and only used to sign the user in as well as obtain a list of possible registered external login providers (e.g., Twitter and Google). The `SignInManager` class is fully documented at http://mng.bz/m2z4.

The `UserManager` provides an API for managing the user with a database or other registered persistence store. Accordingly, it includes a wide range of methods for saving and retrieving user data, including the methods in table 9.7.

Table 9.7 Select `UserManager` methods

Method	Description
`CreateAsync`	Adds a user to the database
`UpdateAsync`	Updates the user details in the database
`DeleteAsync`	Removes the user from the database
`GetUserAsync`	Gets the user corresponding to the `ClaimsPrincipal` passed in to the method
`FindByIdAsync`	Retrieves the user with the specified `Id`
`FindByNameAsync`	Retrieves the user with the specified `Name` claim
`AddClaimAsync`	Adds a claim to the user
`AddToRoleAsync`	Adds the user to the specified `Role`

In addition, there are methods for managing passwords, confirmation tokens, two-factor authentication, lockouts, and so on. The last two methods in the table are more likely to be used as part of an authorization workflow, and we will look at them in a little more detail in the next chapter. Like the `SignInManager`, the `UserManager` is injected as a service wherever it is required. For details on the wide range of other methods and properties on the `UserManager` class, you can refer to the documentation: http://mng.bz/5mYa.

Returning to customizing the UI, first you will modify the Login page, so it no longer features guidance on wiring up external authentication services. Open the scaffolded Login.cshtml file, and locate the `div` element that starts with the following code: